

L'essentiel de Python et de la programmation orientée objet

1. Syntaxe du langage Python

- Identifiants, références, conventions de codage et règles de nommage (PEP 8).
- Blocs, commentaires, docstrings.
- Types de données disponibles, typage dynamique, type hints (PEP 484).
- Variables, affichage formaté (f-strings), portée locale et globale.
- Manipulation des types numériques et des chaînes de caractères.
- Manipulation des listes, tuples et dictionnaires ; sets et compréhensions.
- Lecture et écriture de fichiers, gestionnaires de contexte (`with`).
- Structure conditionnelle `if / elif / else`, expressions ternaires, `match` (Python 3.10+).
- Opérateurs logiques et opérateurs de comparaison.
- Boucles `while` et `for`, interruption d'itérations `break / continue`, clause `else`.
- Fonction `range`, énumération, `zip`.
- Écriture et documentation de fonctions, arguments par défaut, `*args / **kwargs`.
- Lambda expressions, fonctions d'ordre supérieur (`map`, `filter`, `sorted`).
- Générateurs et expressions génératrices.
- Structuration du code en modules et packages.
- Environnements virtuels (`venv`) et gestion des dépendances (`pip`, `requirements.txt`, `pyproject.toml`).

2. Programmation orientée objet

2.1 Principes du paradigme objet

- Définition d'un objet : état, comportement, identité.
- Notions de classe, attributs et méthodes.
- Encapsulation des données, interface publique vs implémentation.
- Communication entre objets, couplage et cohésion.
- Héritage et transmission des caractéristiques d'une classe.
- Association, agrégation, composition entre classes.
- Notion d'interface et de protocole.
- Introduction aux design patterns courants (Strategy, Factory, Observer, Singleton).

2.2 Programmation objet en Python

- Particularités du modèle objet de Python (tout est objet, MRO, duck typing).
- Écriture de classes, instanciation, constructeurs (`__init__`) et destructeurs (`__del__`).
- Attributs de classe vs attributs d'instance.
- Conventions de visibilité (`__protégé`, `__privé`), propriétés et descripteurs.
- Rôle du paramètre `self` et des méthodes de classe / statiques.
- Héritage simple et multiple, méthode `super()`, résolution d'ordre des méthodes.
- Polymorphisme et duck typing en Python.
- Méthodes spéciales (dunder) : `__str__`, `__repr__`, `__eq__`, `__hash__`, `__iter__`, `__enter__`...
- Introspection (`dir`, `type`, `isinstance`, `getattr`) et metaprogrammation légère.
- Implémentation d'interfaces avec `abc.ABC` et `Protocol` (PEP 544).
- Dataclasses (PEP 557) pour les structures de données.
- Gestion des erreurs : exceptions, hiérarchie, exceptions personnalisées, gestionnaires de contexte.
- Bonnes pratiques et design patterns appliqués en Python.

3. Bibliothèque standard et expressions régulières

- Arguments passés sur la ligne de commande (`sys.argv`, `argparse`).
- Modules `sys`, `os`, `os.path`, `pathlib` pour la manipulation du système de fichiers.
- Manipulation de dates et durées avec `datetime`.
- Sérialisation : `json`, `csv`, `pickle`.
- Logging : module `logging`, niveaux, handlers, formatters, configuration.
- Empaquetage et distribution d'une bibliothèque Python.

3.1 Expressions régulières et analyse de logs

- Module `re` : compilation, recherche, substitution.
- Caractères spéciaux, classes de caractères, ancrages, cardinalités.
- Groupes de capture, groupes nommés, lookahead / lookbehind.
- Application à l'analyse de logs HTTP : extraction de timestamps, méthodes, URI, codes de retour.
- Performance et pièges classiques (catastrophic backtracking).

4. Accès aux bases de données relationnelles

- Fonctionnement de la DB-API Python (PEP 249).
- SQLite : prise en main avec le module `sqlite3`.
- Création de schéma, requêtes paramétrées, prévention des injections SQL.
- Transactions, gestion des erreurs, gestionnaire de contexte.

5. Programmation asynchrone et APIs web

5.1 Programmation asynchrone

- Concurrency vs parallélisme, GIL et conséquences.
- Threads et processus : modules `threading` et `multiprocessing` en survol.
- Coroutines, `async` / `await`, boucle d'événements `asyncio`.
- Tâches, `gather`, `wait`, timeouts, annulation.
- Client HTTP asynchrone avec `httpx` ou `aiohttp` pour appeler des équipements REST.
- Bonnes pratiques : éviter les opérations bloquantes, structurer les services I/O-bound.

5.2 Création d'APIs web légères

- Comparatif Flask vs FastAPI : philosophie, performance, validation, documentation.
- FastAPI : routes, paramètres, modèles Pydantic, validation automatique.
- Documentation OpenAPI / Swagger générée automatiquement.
- Endpoints synchrones et asynchrones, dépendances, middleware.
- Gestion des erreurs et codes HTTP.
- Démarrage avec `uvicorn`, structuration d'un projet.
- Survol de Flask pour les cas plus simples.

6. Outils QA, tests et IA dans le développement

6.1 Qualité du code

- Outils d'analyse statique : Pylint, Flake8, Ruff, mypy.
- Analyse des comptes rendus : types de messages, avertissements, erreurs.
- Formatage automatique : Black, isort.
- Extraction automatique de documentation : docstrings, Sphinx.

6.2 Débogage et profiling

- Débogueur `pdb` : exécution pas à pas, points d'arrêt, analyse post-mortem.
- Intégration avec les IDE (VS Code, PyCharm).
- Profiling : `cProfile`, `timeit`, identification des goulets d'étranglement.

6.3 Tests unitaires et TDD

- Le développement piloté par les tests (TDD) : cycle red / green / refactor.
- Modules de tests Python : `unittest` et `pytest` (recommandé).
- Fixtures, paramétrage, mocks, patches.
- Tests d'API : TestClient FastAPI, requêtes simulées.
- Automatisation : agrégation de tests, exécution en CI.

- Couverture de code : `coverage.py`, intégration dans la CI.
- Survol de l'intégration continue (GitLab CI, GitHub Actions).

6.4 IA et bonnes pratiques de développement

- Cadrer un assistant IA comme un développeur junior : revue, validation, contexte fourni.
- Documenter les conventions de l'équipe pour les rendre exploitables par un LLM.
- Préserver le TDD : écrire le test avant de demander l'implémentation à l'IA.
- Prompts efficaces pour générer du code testé, des fixtures, des cas limites.
- Risques : hallucinations, dépendances fantômes, code superficiellement correct.
- Revue de code assistée par IA : forces et limites.

7. Interfaçage Python / C

- Présentation du module `ctypes`.
- Chargement d'une bibliothèque C ou d'une DLL Windows.
- Déclaration des signatures de fonctions, types `argtypes` / `restype`.
- Passage de structures, pointeurs, tableaux, callbacks.
- Gestion mémoire, pièges classiques (durée de vie des buffers).
- Présentation rapide de Cython et des alternatives modernes (cffi, pybind11).
- Compilation d'un script Python en extension C avec Cython.
- Utilisation des annotations pour améliorer les performances.
- Profiling avant / après pour mesurer l'apport.